Team Project – Iteration 1 Deliverable

Smart Study Planner

Group Members: Katherine Toro Villanueva, Daniel Thompson, Zinah Shain

Date: 2026-03-05

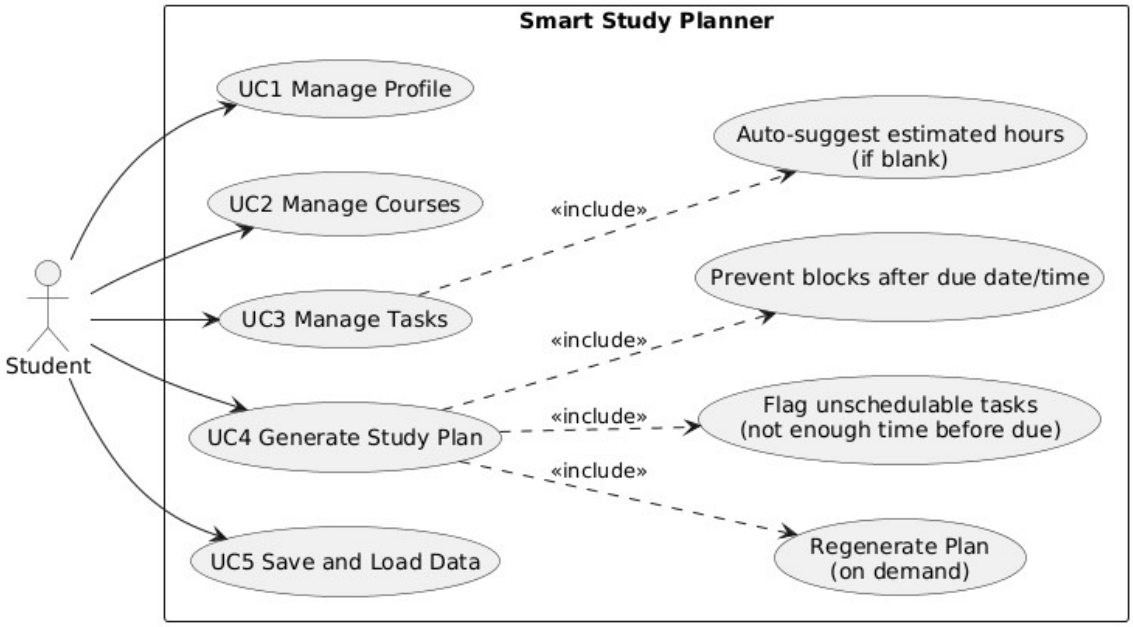
Version 1 Requirements Implemented

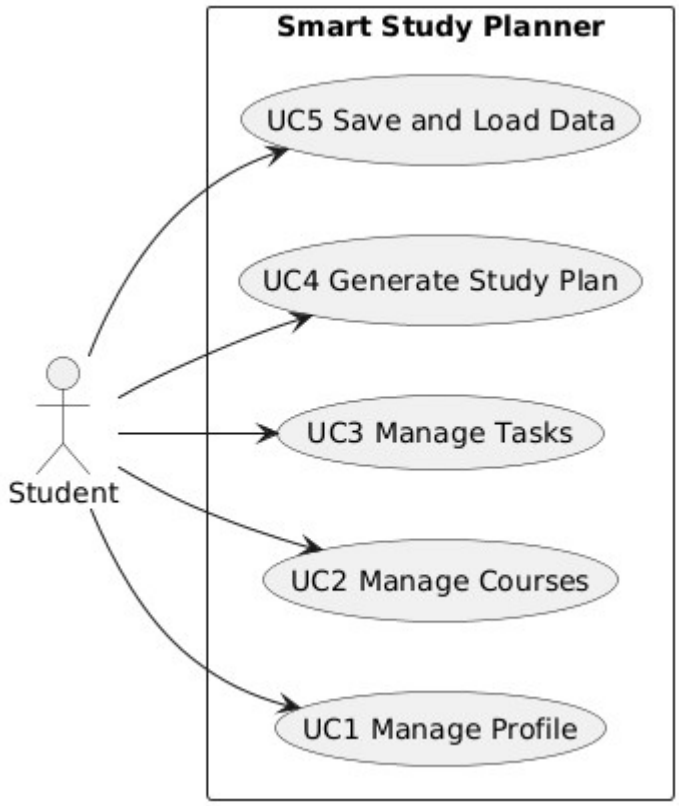
- FR1 Profile CRUD (simplified: edit single local profile).
- FR2 Courses CRUD.
- FR3 Tasks CRUD (title, type, due date/time, estimated hours, priority, notes) and association to course.
- FR4 Set study availability per day of week (0–12 hours/day).
- FR5 Task list view (table); supports basic viewing (sorting can be done by clicking table headers).
- FR6 Mark tasks Not Started / In Progress / Completed.
- FR7 Generate study plan for a selected date range (default next 7 days).
- FR8 Enforce: no study block scheduled after a task due date/time; flag unschedulable tasks.
- FR9 Regenerate plan on demand; completed tasks contribute 0 remaining blocks.
- FR10 Save/Load all data to/from a local file.
- FR11 Auto-suggest estimated study hours when blank (max(0.5, credit_hours × 1.0)).

Actors

Primary Actor: Student (single user of the desktop application).

1. Use Case Diagram





2. Use Cases (Two-Column Format)

Use Case: UC1 Manage Profile

Field	Description
Primary Actor	Student
Goal	Create or update the local user profile settings.
Preconditions	Application is running.
Trigger	Student opens the Profile tab and edits fields.
Main Success Scenario	1) Student enters name, weekly goal hours, and preferred block length. 2) Student clicks Apply. 3) System validates input. 4) System saves the profile settings locally.
Extensions / Alternate Flows	A1 Invalid input (blank name, out-of-range hours/minutes): System shows validation error; no changes are saved.

Use Case: UC2 Manage Courses

Field	Description
Primary Actor	Student
Goal	Create and delete courses.
Preconditions	Application is running.
Trigger	Student clicks Add or Delete in Courses tab.
Main Success Scenario (Add)	1) Student clicks Add. 2) System shows course form. 3) Student enters course name, credit hours, and optional tags/notes. 4) Student clicks Save. 5) System stores course and updates course list.
Main Success Scenario (Delete)	1) Student selects a course. 2) Student clicks Delete. 3) System asks for confirmation. 4) Student confirms deletion. 5) System removes the course from the list.
Extensions / Alternate Flows	A1 Invalid course data: System rejects input and shows error message.

Use Case: UC3 Manage Tasks

Field	Description
Primary Actor	Student
Goal	Create, update status, and delete tasks.
Preconditions	At least one course exists to attach a task to.
Trigger	Student clicks Add / Set Status / Delete in Tasks tab.
Main Success Scenario (Add)	1) Student clicks Add. 2) System shows task form. 3) Student selects course and enters task title, type, due date, estimated hours, and priority. 4) Student clicks Save. 5) System stores task and updates task list.
Main Success Scenario (Set Status)	1) Student selects a task. 2) Student chooses a status (Not Started / In Progress / Completed). 3) System updates the task status.
Main Success Scenario (Delete)	1) Student selects a task. 2) Student clicks Delete. 3) System asks for confirmation. 4) Student confirms deletion. 5) System removes the task.
Extensions / Alternate Flows	A1 Invalid task data (missing title or invalid due date): System shows error and does not save task.

Use Case: UC4 Generate Study Plan

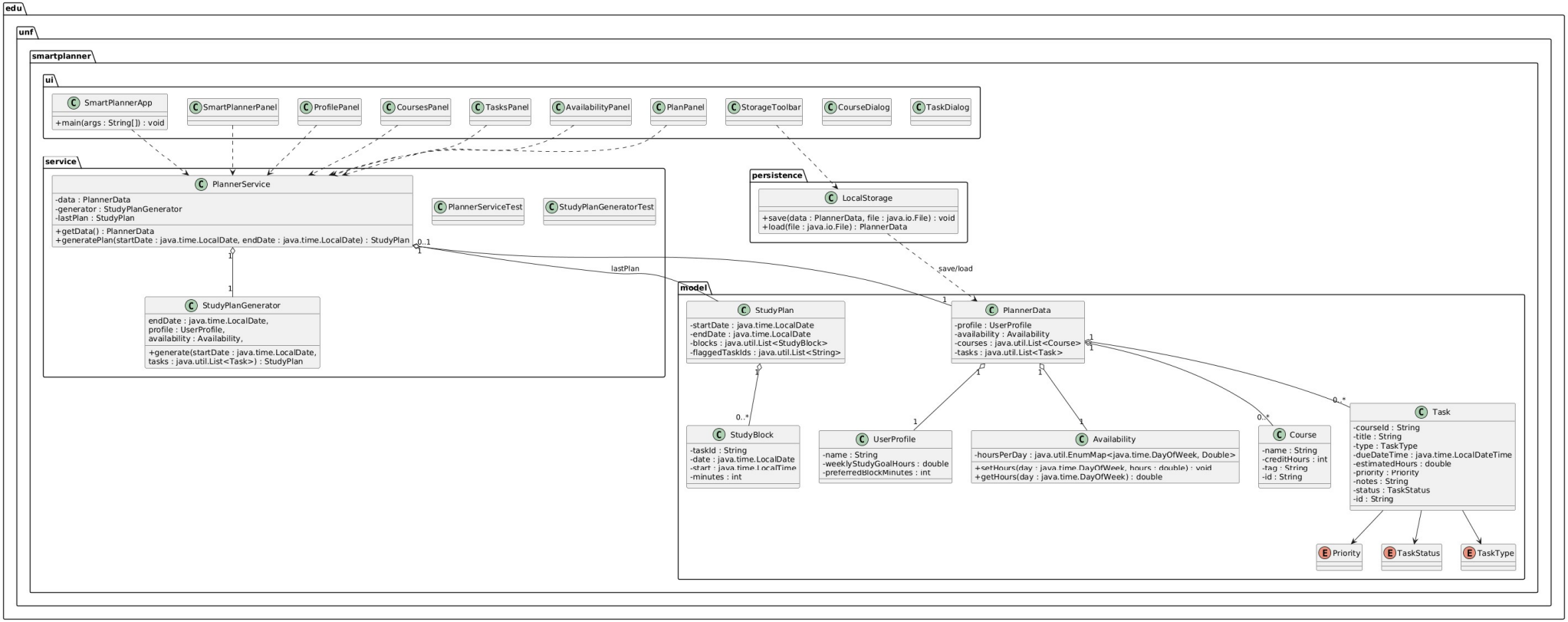
Field	Description
Primary Actor	Student
Goal	Automatically generate a weekly study schedule based on tasks and availability.
Preconditions	Tasks and availability have been defined.
Trigger	Student clicks “Generate Plan”.
Main Success Scenario	1) Student clicks Generate Plan. 2) System retrieves tasks and availability. 3) System prioritizes tasks based on due date and priority. 4) System allocates study blocks into available time slots.

	5) System displays generated study schedule.
Extensions / Alternate Flows	A1 Insufficient availability: System flags tasks that cannot be scheduled before the due date.

Use Case: UC5 Save and Load Data

Field	**Description**
Primary Actor	Student
Goal	Save planner data and reload it later.
Preconditions	Application is running.
Trigger	Student clicks Save or Load.
Main Success Scenario (Save)	1) Student clicks Save. 2) System writes profile, availability, courses, and tasks to a file. 3) System confirms successful save.
Main Success Scenario (Load)	1) Student clicks Load. 2) Student selects saved file. 3) System loads profile, availability, courses, and tasks. 4) System updates the interface with loaded data.
Extensions / Alternate Flows	A1 File error: System displays error message if file cannot be loaded or saved.

3. UML Class Diagram



4. Implementation Overview (Version 1)

Version 1 implements the core planner functionality including profile creation, course and task management, and automatic study plan generation.

Implemented Features:

- Create and edit user profile
- Add courses
- Add tasks with due dates and priorities
- Define weekly study availability
- Generate study schedule
- Mark tasks as completed

5. GitHub Commit Log

Paste a screenshot or copy of the GitHub commit log here showing contributions from each team member.

6. How to Run the System

1. Download the project ZIP file and extract it.

2. Open the folder “Smart Study Planner Github” in an IDE (IntelliJ recommended; Eclipse also works).

3. Locate the main class: edu.unf.smartplanner.ui.SmartPlannerApp

4. Run SmartPlannerApp.main() from the IDE.

5. Use the GUI tabs/panels to manage profile, courses, tasks, availability, and generate the study plan.

6. Use the Save/Load buttons in the UI to save data locally and load it back.

7. Features, Design Quality, and Programming Style

Features Implemented

- Implemented UC1–UC5: profile management, course management, task management, plan generation for a date range (default next 7 days), and local save/load.
- Study plan generation enforces: no blocks scheduled after a task's due date/time, and flags tasks that cannot be fully scheduled before the due date.
- Task status updates (In Progress/Completed) are supported, and regeneration removes remaining blocks for completed tasks.

Design Quality

- The code is organized by responsibility into packages: model (domain objects like Task, Course, StudyPlan, PlannerData), service (business logic like PlannerService, StudyPlanGenerator), persistence (local storage via LocalStorage), and ui (Swing panels/dialogs and SmartPlannerApp).
- UI components call into PlannerService instead of directly implementing scheduling/storage logic, keeping concerns separated.

Programming Style

- Classes and methods use clear, descriptive names aligned with the domain.
- Enums (Priority, TaskStatus, TaskType) improve readability and reduce invalid states.

Methods are kept focused on single responsibilities where possible, and code uses

consistent formatting.

Appendix A. README.md (How to Run in Code Zip)

Smart Study Planner (Iteration 1)

Steps

1. Download and extract the project zip.

2. Open the folder `Smart Study Planner Github` in your IDE.

3. Run the main class: `edu.unf.smartplanner.ui.SmartPlannerApp`

4. Use the GUI tabs/panels to manage profile, courses, tasks, availability, generate/view the plan, and save/load data.

Running Unit Tests

- Tests are located in: `src/test/java/edu/unf/smartplanner/service/`

- Run tests using your IDE's JUnit runner.